Final Report for Battery Management System (BMS)

Sponsored by Viking Motorsports (VMS)

Portland State University — Maseeh College of Engineering & Computer Science — Department of Electrical & Computer Engineering

Authors:

- Jose Ramirez (ramirezjosejr378@gmail.com)
- Hussein Khodor
- David Fransis
- Fangdong Yuan

Date: June 13, 2026

ECE Capstone — Senior Project Development

Portland State University — Maseeh College of Engineering & Computer Science — Department of Electrical & Computer Engineering **1**
Executive Summary 4
Background 5
BMS Role in the EV Platform 5
BMS Architecture Approach 5
Voltage Domains and Isolation 6
Battery Monitoring Requirements 6
Communication Background 6
Monitoring and Fault Detection Logic 7
Requirements 7
Stakeholders 7
Concept of Operation 7
Initial Sponsor Requirements 8
Requirement Changes During the Project 8

Final Requirements 8
Must-Have Requirements 9
Should-Have Requirements 9
May-Have Requirements 9
Summary 9
Objectives and Deliverables 9
Project Objective 9
Original Expected Deliverables 10
Hardware Objectives and Deliverables 11
Software Objectives and Deliverables 11
Change in Deliverable Scope 11
Design 12
1. Design Overview 12
2. System Architecture 13
3. Board-Level Summary 13
4. Battery Management Unit Design 14
4.1 Main Control Function 14
4.2 Power Input and Regulation 14
4.3 BQ79600-Q1 Bridge Circuit 14
4.4 CAN Communication Circuit 15
4.5 Debug and Board-to-Board Interfaces 15
5. Cell Monitoring Unit Design 15
5.1 Cell Voltage Inputs 15
5.2 Cell Balancing Support 15
5.3 Temperature Sensing 16
5.4 CMU Communication Path 16
6. Isolated Pack Current Sense Board Design 16
6.1 Current Measurement 16
6.2 Isolated I2C Communication 16
6.3 Isolated ALERT Signal 16
6.4 Isolated Power Domain 17
6.5 Debugging Features 17
7. Communication Design 17
8. Power and Isolation Design 18
9. Monitoring and Fault Detection Logic 19
10. Firmware Design 20
11. Design Rationale 20
12. Design Limitations and Verification Needs 21
Test Plan Overview 21
Results 21
Project Result Summary 22

Deliverables Achieved 22
Test Results and Design Notes 22
Requirement Satisfaction 23
Overall Result 23
Post Mortem 24
What Worked 24
What Did Not Work 24
What We Would Do Differently 24
What We Wish We Knew Earlier 24
Future Work 25
Project Resources 25
Conclusion 25
Appendix A: Tooling 26
A.1 Final Prototype Hardware Tools 26
A.2 Final Prototype Software Tools 26
A.3 Zephyr Toolchain Installation 27
Appendix B: Developer's Manual 27
B.1 Developer Overview 27
B.2 System Architecture 28
B.3 Hardware Configuration 28
B.3.1 Battery Pack Configuration 28
B.3.2 BQ79616EVM 6S Configuration 29
B.3.3 BQ79600EVM to STM32 Nucleo UART Connection 29
B.3.4 INA238EVM to STM32 Nucleo I2C Connection 30
B.3.5 Shunt Configuration 30
B.3.6 CAN Transceiver Connection 30
B.3.7 Power Distribution 31
B.4 Firmware Architecture 31
B.5 Firmware Initialization Flow 32
B.6 Building and Flashing Firmware 32
B.7 CAN Message Summary 33
Appendix C: User's Manual 33
C.1 System Overview 33
C.2 Prototype Connections 34
C.3 Hardware Connection Procedure 34
C.3.1 Battery Pack and BQ79616EVM 34
C.3.2 BQ79600EVM and BQ79616EVM 35
C.3.3 STM32 Nucleo Connections 35
C.3.4 INA238EVM and Shunt 35
C.3.5 CAN Connection to External ECU 35
C.4 Firmware Build and Flash Procedure 35

C.4.1 Build the Firmware 35
C.4.2 Flash the Firmware 36
C.4.3 Serial Monitor Configuration 36
Appendix D: Test Plan 36
D.1 Top Down Testing 36
D.1.1 Purpose 36
D.1.2 Equipment Needed 36
D.1.3 Top Down Test Steps 37
D.1.4 Post-Test Teardown 37
D.2 Bottom-Up Testing 37
D.2.1 Power Rail Test 37
D.2.2 Cell Sense Harness Test 37
D.2.3 Cell Sense Harness Test 38
D.2.4 INA238 Current Measurement Test 38
D.2.5 CAN Communication Test 38

Executive Summary

Viking Motorsports is developing a Formula SAE Electric race car for the 2027 competition season. One of the major systems needed for this vehicle is a Battery Management System, or BMS. The BMS protects the battery pack by monitoring cell voltages, pack current, and battery temperature. It also reports battery information and fault conditions to the vehicle controller.

Our capstone project focused on designing the core electrical hardware for this BMS. The original sponsor goal was to create a functional BMS prototype capable of basic monitoring, balancing, CAN communication, fault detection, and future integration with the vehicle ECU. As the project developed, the scope became more focused because the full vehicle battery system was still being defined. Instead of completing a fully vehicle-ready BMS, we focused on creating a modular hardware design that future Viking Motorsports teams can test, program, and integrate into the 2027 EV platform.

The final design is organized into three major circuit boards: the Battery Management Unit, the Cell Monitoring Unit, and the Shunt / Current Sense Board. The Battery Management Unit acts as the main controller board. It includes the STM32 microcontroller, power regulation, CAN communication, programming/debug access, and the BQ79600 bridge used to communicate with the battery monitoring system. The Cell Monitoring Unit uses the BQ79616 battery monitor to measure individual cell voltages, support temperature sensing, and provide cell balancing connections. The Shunt / Current Sense Board uses the INA238 current monitor to measure pack current through an external shunt resistor and sends that data back to the BMU through isolated I2C communication.

A major design focus was electrical isolation. The BMS connects to battery-side circuits while also communicating with low-voltage vehicle electronics. To reduce risk, the current sensing board uses isolated power, isolated I2C communication, and an isolated ALERT signal. The design also separates the battery monitoring communication path from the controller side through the BQ79600/BQ79616

interface.

By the end of the project, we produced a documented first-stage BMS hardware design. The main successes were defining the system architecture, selecting the major ICs, creating the BMU/CMU/shunt schematics, organizing the design into modular subsystems, and preparing the design for future firmware and PCB work. The main limitation was that the project did not reach a fully assembled, tested, vehicle-ready prototype. Future work should focus on PCB layout review, board fabrication, firmware development, sensor validation, CAN communication testing, fault threshold testing, and integration with the Viking Motorsports ECU and accumulator system.

This project gives the next team a clear starting point for the 2027 EV BMS. They should be able to use the schematics, BOMs, design notes, and tooling documentation to continue the design instead of starting over.

Background

BMS Role in the EV Platform

A Battery Management System (BMS) is one of the main safety and monitoring systems in an electric vehicle. Its job is to monitor the battery pack, protect the cells from unsafe operating conditions, and report battery status to the rest of the vehicle. For the Viking Motorsports EV platform, the BMS must support the traction battery system by measuring cell voltage, pack current, and temperature. It must also help detect fault conditions such as overvoltage, undervoltage, overcurrent, and overtemperature.

This project focuses on creating a BMS foundation for the 2027 Viking Motorsports EV race car. Since the full accumulator and vehicle electronics are still being developed, the BMS needs to be modular, testable, and expandable rather than fixed to only one final battery layout.

BMS Architecture Approach

A BMS can be organized in several ways. A centralized BMS places most sensing, control, and communication circuits on one main board. This can be simpler for small battery systems, but it becomes harder to scale when the battery pack has many series cells, multiple sensing locations, or strict isolation requirements. A distributed BMS places monitoring circuits closer to the battery modules and sends data back to a main controller. This can improve scalability and reduce long cell-sense routing, but it requires clear communication and isolation planning between boards.

For this project, the BMS uses a modular architecture that is closer to a distributed system than a single centralized board. The system is divided into three major subsystems: the Battery Management Unit (BMU), the Cell Monitoring Unit (CMU), and the Isolated Pack Current Sense Board, referred to as the Shunt Board in this report.

The BMU acts as the main controller and vehicle communication hub. The CMU handles battery-side cell voltage and temperature sensing. The Shunt Board measures pack current through an external shunt resistor and sends current data back to the BMU through isolated communication.

This architecture is useful for the VMS EV platform because the final accumulator configuration may continue to change. A modular structure allows the system to scale more easily, allows each subsystem to be tested independently, and keeps high-voltage sensing functions separated from the low-voltage controller side.

Voltage Domains and Isolation

One of the most important concepts in this project is the separation between the high-voltage battery system and the low-voltage vehicle electronics. The accumulator and cell monitoring circuits connect close to the battery pack, while the main controller, ECU communication, and debugging circuits operate on low-voltage logic rails. These areas cannot always share the same ground or voltage reference safely.

Because of this, the BMS architecture must consider voltage domains and isolation from the beginning. The low-voltage controller side includes the STM32, CAN interface, programming connector, and logic-level circuits. The battery-side sensing circuits include the cell monitoring inputs and the current sensing circuitry. If these domains are connected incorrectly, a wiring mistake or ground reference difference could damage the controller, communication circuits, or vehicle electronics.

Isolation helps reduce this risk. The current sensing path uses isolated communication so current data can return to the BMU without directly tying the measurement-side ground to the controller-side ground. The isolated power domain also allows the current sensing circuit to operate with its own ground reference. This is why the system separates the BMU low-voltage side from the isolated current sensing side.

Battery Monitoring Requirements

The BMS must monitor several battery conditions at the same time. Cell voltage monitoring is required because lithium-based cells must remain within a safe voltage range during charging, discharging, and vehicle operation. If one cell becomes overcharged or over-discharged, the entire pack can become unsafe or damaged.

Temperature monitoring is also required because battery cells can become unsafe if they operate outside their allowed thermal range. Current measurement is needed to understand how much current flows into or out of the pack, detect overcurrent events, and support future diagnostics.

Cell balancing is another important BMS function. In a series battery pack, cells can slowly drift apart in voltage over time. Balancing helps reduce these voltage differences so the pack can operate more safely and consistently. For this project, the design provides hardware support for balancing, while the balancing strategy, firmware logic, and thermal behavior still need to be verified before use with a real accumulator.

Communication Background

Communication is a major part of the BMS architecture. The BMS must collect data from battery-side sensing circuits and send useful battery information to the vehicle ECU. Different parts of the system use different communication methods because they have different electrical and safety requirements.

The cell monitoring path uses a battery monitor daisy-chain structure. This is more scalable than routing every cell directly to the STM32. A daisy-chain approach allows the cell monitoring system to collect data from battery monitoring ICs and pass that information back to the BMU through a structured communication path.

The pack current measurement path uses isolated communication. Since the Shunt Board is connected near the battery current path, it should not directly share the same ground reference as the BMU controller side. Isolated I2C allows the current monitor to send current, voltage, and power data back to the STM32 while maintaining electrical separation.

The BMU communicates with the vehicle ECU over CAN. CAN is commonly used in automotive and embedded vehicle systems because it supports reliable communication between controllers. In this BMS, CAN allows the BMU to send battery data, fault status, and future diagnostic information to the rest of the vehicle.

Monitoring and Fault Detection Logic

The BMS follows a layered monitoring logic. First, the CMU measures individual cell voltages and battery temperatures. Second, the Shunt Board measures pack current through the external shunt resistor. Third, the BMU collects voltage, temperature, and current data from these subsystems. Fourth, the STM32 on the BMU compares the measured values against safety limits. Finally, the BMU reports battery status and fault information to the vehicle ECU over CAN.

This order is important because the BMU does not directly measure every battery condition by itself. Instead, the BMS uses dedicated sensing subsystems to collect battery data, then uses the BMU as the central decision-making and communication board. This structure makes the project easier to understand: the CMU monitors cell-level conditions, the Shunt Board monitors pack-level current, and the BMU combines this information into system-level battery status and fault reporting.

Suggested figure: insert a high-level block diagram showing the accumulator, CMU, BMU, Shunt Board, ECU, CAN communication path, battery monitor daisy-chain path, and isolated current sensing path.

Requirements

The sponsor asked us to design a Battery Management System for the PSU Viking Motorsports 2027 EV race car. At a high level, the BMS needed to support the future Formula SAE Electric accumulator by monitoring the battery pack, detecting unsafe conditions, and communicating battery status to the vehicle electronics.

Stakeholders

The main stakeholders for this project were:

- **Viking Motorsports:** needs a BMS foundation for the 2027 Formula SAE Electric vehicle. Including schematics, part selections, and design documentation that can be tested and continued, a BMS design that can eventually fit within the accumulator, vehicle packaging constraints, battery data and fault information from the BMS.
- **Future capstone and VMS teams:** need a design that is understandable, reproducible, and expandable.
- **Faculty advisor and sponsor contacts:** need the project to follow the safety and documentation expectations of the capstone and FSAE EV environment.

Concept of Operation

The BMS is expected to operate as the battery pack's monitoring and reporting system. During operation, the BMS reads battery measurements, checks those measurements against safe limits, and reports the battery state to the vehicle ECU. If a critical fault occurs, the BMS should support a safe response through a shutdown or interlock path.

The expected system behavior is:

1. Measure battery cell voltages, pack current, and battery temperature.
2. Compare those measurements against defined safety limits.
3. Detecting unsafe battery conditions.
4. Report battery data and fault states to the ECU over CAN.
5. Support a safe shutdown or interlock response for critical faults.

Initial Sponsor Requirements

The original sponsor requirements described a functional BMS prototype for the Viking Motorsports Formula SAE Electric vehicle. The initial requirements were:

- Monitor all cell voltages.
- Monitor pack temperature.
- Measure battery current.
- Detects overvoltage, undervoltage, overcurrent, and overtemperature conditions.
- Support cell balancing.
- Communicate with the ECU over CAN bus.
- Operate within Formula SAE Electric voltage and isolation expectations.
- Include a fail-safe shutdown or interlock function for critical faults.
- Support future testing, diagnostics, and vehicle integration.

The original project description also listed additional desired features, including onboard data logging, modular scaling for different battery pack sizes, self-diagnostic startup checks, fault recovery routines, a local diagnostic interface, USB debugging, and possible wireless telemetry.

Requirement Changes During the Project

The requirements changed because the full 2027 EV system was still being defined during the capstone cycle. The vehicle battery architecture, ECU integration plan, and final accumulator details were not fully fixed at the beginning of the project. Because of that, the team could not treat the BMS as a finished vehicle-ready product.

The biggest change was the project scope. The original goal included a functional BMS prototype with monitoring, balancing, firmware, CAN communication, test results, and safety documentation. As the project progressed, the scope shifted toward creating a first-stage BMS hardware design that future teams could continue. This changed the final requirement from “complete and validate a vehicle-ready BMS” to “create a modular BMS design foundation with clear documentation and a path toward testing.”

Another major change was the emphasis on modularity. Instead of treating the BMS as one large circuit, the system needed to be broken into major subsystems so the team could divide the work and future teams could test each section separately. This made modular board-level design a final requirement rather than just a design preference.

Isolation also became a more important requirement as the design developed. The BMS needed to connect battery-side measurement circuits to low-voltage controller electronics. Because these sections can have different voltage references and safety risks, the final requirements needed to include isolated communication and separated power domains where appropriate.

Final Requirements

By the end of the project, the final requirements were organized into must-have, should-have, and

may-have categories.

Must-Have Requirements

The BMS must:

- Monitor individual battery cell voltages.
- Monitor battery temperature.
- Measure battery pack current.
- Detect overvoltage, undervoltage, overcurrent, and overtemperature conditions.
- Support cell balancing at the hardware level.
- Communicate battery data and fault status to the ECU over CAN.
- Include a path for safe shutdown or interlock behavior during critical faults.
- Use electrical isolation where needed between battery-side sensing and low-voltage control electronics.
- Use a modular architecture that separates major BMS functions into testable subsystems.
- Include documentation that allows future teams to continue the design.

Should-Have Requirements

The BMS should:

- Support startup self-diagnostics.
- Support fault recovery routines when safe.
- Support data logging for testing and diagnostics.
- Scale to future battery pack configurations.
- Include test points or debug access for important power and communication signals.
- Use parts and interfaces that future teams can source, assemble, and test.

May-Have Requirements

The BMS may:

- Include a local display or diagnostic interface.
- Support USB debugging for development.
- Support wireless telemetry for testing.
- Include more advanced data logging or analysis features.
- Expand balancing features depending on time, budget, and final accumulator needs.

Summary

The final requirements focused on creating a safe, modular, and expandable BMS foundation for the Viking Motorsports 2027 EV race car. The project started with the goal of a functional prototype, but the requirements changed as the team learned more about the vehicle timeline and system dependencies. The final requirements prioritized core battery monitoring, fault detection, ECU communication, isolation, and documentation so future teams can continue development without starting over.

Objectives and Deliverables

Project Objective

The objective of this project was to move the Viking Motorsports Battery Management System (BMS) from a broad system concept toward a practical design package that future teams can build on. The sponsor's long-term goal is a BMS that can support the 2027 Formula SAE Electric vehicle by monitoring the traction battery pack, detecting unsafe operating conditions, and communicating battery status to the vehicle ECU.

The original project goal was to develop a functional BMS prototype capable of basic monitoring, balancing support, fault detection, and ECU communication. The expected system functions included cell voltage monitoring, pack current measurement, temperature sensing, overvoltage and undervoltage detection, overcurrent and overtemperature detection, cell balancing support, CAN communication, and safety-related shutdown or fault reporting behavior.

Original Expected Deliverables

At the beginning of the project, the sponsor expected the team to deliver both hardware and software work that could support future vehicle integration. The ideal final result was a working BMS prototype with supporting documentation, firmware, PCB files, test results, and a final demonstration.

Expected Deliverable	Purpose
Hardware schematics	Define the BMS circuits for cell monitoring, current sensing, power regulation, isolation, and communication.
PCB design files	Provide board files that future teams could fabricate, assemble, and revise.
Firmware source code	Allow the BMU microcontroller to collect battery data, check faults, and communicate with the ECU.
CAN communication support	Send battery data and fault status to the vehicle control system.
Test plan and test results	Show how the BMS subsystems should be tested and what level of validation was completed.
Project documentation	Explain the design decisions, setup process, limitations, and future work.

Final demonstration	Demonstrate the core BMS functions if hardware and firmware integration were completed in time.

Hardware Objectives and Deliverables

The main hardware objective was to design the core electrical architecture of the BMS. This included the controller board, the cell monitoring board, and the isolated current sensing board. The hardware work focused on a modular structure so each subsystem could be reviewed, tested, and improved separately.

The planned hardware deliverables included:

- **Battery Management Unit (BMU)** schematic and design files.
- **Cell Monitoring Unit (CMU)** schematic and design files.
- **Isolated Pack Current Sense Board**, referred to as the **Shunt Board**, schematic and design files.
- Major IC selections for monitoring, communication, current sensing, and isolation.
- Board-to-board interface planning.
- Power-domain and isolation planning.
- BOM review and component selection support.
- PCB layout work where completed.

The final hardware deliverable was a first-stage BMS hardware design package for the BMU, CMU, and Shunt Board. These designs provide the foundation for PCB fabrication, assembly, subsystem bring-up, firmware integration, and future vehicle-level testing.

Software Objectives and Deliverables

The software objective was to support communication, data collection, and fault handling for the BMS. The software was expected to run on the STM32 microcontroller on the BMU and eventually communicate with the CMU, Shunt Board, and vehicle ECU.

The planned software deliverables included:

- STM32 firmware source code.
- Communication setup for the BMS hardware.
- CAN communication support for ECU integration.
- Sensor reading routines for voltage, temperature, and current data.
- Fault detection logic.
- Debugging support for firmware bring-up.

The software work remained focused on the structure needed for future firmware integration. Full firmware validation, complete CAN message definition, and full system-level testing still need to be completed after hardware bring-up.

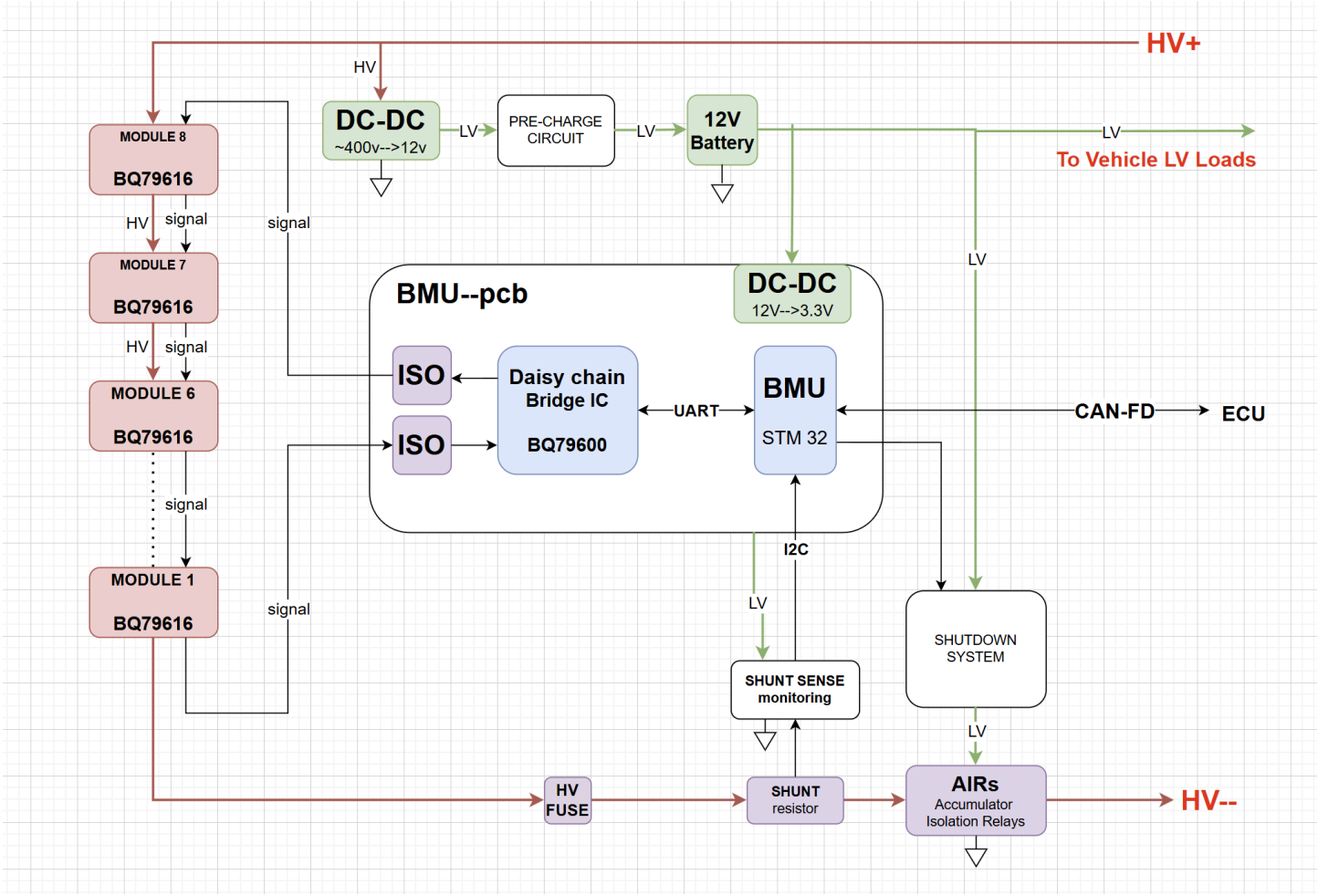
Change in Deliverable Scope

The deliverables changed because the project required more system-level definition than originally expected. The full EV platform, accumulator design, and integration plan were still being developed, so the team had to make major decisions about architecture, part selection, isolation, board separation,

communication interfaces, and power domains before a reliable prototype could be built.

Because of this, the final deliverable shifted from a fully assembled and validated vehicle-ready BMS to a documented first-stage BMS design foundation. This change does not remove the long-term goal of a working vehicle-ready BMS. Instead, it defines this capstone project as the first major design step toward that goal.

By the end of the project, the intended outcome was a BMS design package that future Viking Motorsports teams could understand, reproduce, and continue. The project created the structure needed for battery monitoring, current sensing, communication, isolation, and future safety logic, while leaving full prototype validation and vehicle integration as future work.



Design

1. Design Overview

The final Battery Management System (BMS) design uses a **modular three-board architecture**. The system is divided into:

1. **Battery Management Unit (BMU)**
2. **Cell Monitoring Unit (CMU)**
3. **Isolated Pack Current Sense Board**, referred to as the **Shunt Board** in this report

This structure was chosen because the 2027 Viking Motorsports EV platform is still developing. A modular design allows the system to be tested subsystem by subsystem and gives future teams more flexibility if the accumulator voltage, series cell count, connector layout, or ECU communication requirements change.

At a high level, the **CMU monitors cell-level conditions**, the **Shunt Board monitors pack-level current**, and the **BMU collects the data, checks for faults, and communicates with the vehicle ECU over CAN**.

2. System Architecture

The BMS is organized around three main functions: **cell monitoring**, **current sensing**, and **vehicle communication**.

Data Path	Architecture and Purpose
Cell Voltage and Temperature Monitoring	**Battery cells → CMU → BQ79616-Q1 → BQ79600-Q1 bridge → STM32 on BMU**. This path collects individual cell voltages and temperature readings. The CMU handles battery-side sensing, while the BMU receives the data through the BQ79600-Q1 bridge.
Pack Current Measurement	**External shunt resistor → Shunt Board → INA238-Q1 → ISO1540-Q1 isolated I2C → STM32 on BMU**. This path measures pack current through the external shunt resistor. The Shunt Board sends current, voltage, and power data to the BMU through isolated communication.
Fault and ECU Communication	**Voltage / temperature / current data → STM32 fault logic → CAN transceiver → vehicle ECU**. The BMU compares measured values against safety limits and sends battery status and fault information to the ECU over CAN.

The system separates the **battery-side sensing circuits** from the **low-voltage controller circuits**. This separation is important because the cell monitoring and current sensing circuits may operate at different electrical potentials than the STM32, CAN transceiver, and debugging hardware. The design uses isolated communication and isolated power where needed to reduce the risk of damaging the low-voltage electronics.

Suggested figure: insert a system block diagram showing the accumulator, CMU, BMU, Shunt Board, ECU, CAN path, battery monitor daisy-chain path, and isolated current sensing path.

3. Board-Level Summary

Board	Main Components	Main Function	Connected To
BMU	STM32 microcontroller, BQ79600-Q1 bridge, CAN transceiver, power regulators	Main control, data collection, fault logic, CAN communication, power regulation	CMU, Shunt Board, ECU
CMU	BQ79616-Q1 battery monitor, cell input filters, balancing circuits, NTC thermistor inputs	Cell voltage monitoring, temperature sensing, balancing support	Battery cells, BMU
Shunt Board	INA238-Q1, ISO1540-Q1, ISO7710-Q1, isolated DC-DC converter	Isolated pack current measurement	External shunt resistor, BMU

4. Battery Management Unit Design

The **Battery Management Unit (BMU)** is the main controller board of the BMS. It contains the STM32 microcontroller, power regulation circuits, CAN communication circuit, BQ79600-Q1 bridge circuit, debug/programming connector, and board-to-board connectors.

4.1 Main Control Function

The STM32 is the central controller on the BMU. It collects battery data from the CMU and the Shunt Board, checks the data against safety limits, sets fault flags, and sends battery information to the ECU.

The STM32 is responsible for:

- Initializing communication interfaces.
- Reading cell voltage and temperature data from the CMU.
- Reading current data from the Shunt Board.
- Monitoring fault signals.
- Comparing measurements against safety thresholds.
- Sending battery status and fault information to the ECU over CAN.

4.2 Power Input and Regulation

The BMU receives a **12 V low-voltage input**. The power regulation section converts this input into lower voltage rails used by the controller and communication circuits, including **5 V** and **3.3 V** rails.

The **3.3 V rail** powers the STM32 and low-voltage logic circuits. Input protection components were included to make the board more robust during bench testing and future vehicle integration.

4.3 BQ79600-Q1 Bridge Circuit

The BMU includes the **BQ79600-Q1 bridge circuit**. This device connects the STM32 to the BQ79616-Q1 cell monitoring path on the CMU.

This bridge is important because the BQ79616-Q1 is designed for battery stack monitoring and uses a daisy-chain communication structure. The BQ79600-Q1 allows the STM32 to communicate with the battery monitoring system without directly measuring every cell voltage itself.

4.4 CAN Communication Circuit

The BMU includes a **CAN-FD physical layer interface** using a CAN transceiver. This allows the BMU to communicate with the vehicle ECU through **CANH** and **CANL**.

The CAN circuit includes a connector and termination option so the board can be tested with a CAN analyzer or integrated into the vehicle communication network later.

4.5 Debug and Board-to-Board Interfaces

The BMU includes an **STDC14 / SWD connector** for STM32 programming and debugging.

The BMU also connects to the CMU and the Shunt Board. The CMU connection carries the battery monitor communication path. The Shunt Board connection provides the BMU-side low-voltage interface and receives isolated current data and the ALERT signal.

Because of these connections, the BMU acts as the main collection point for **cell voltage, temperature, current, and fault information**.

5. Cell Monitoring Unit Design

The **Cell Monitoring Unit (CMU)** measures individual battery cell voltages and battery temperatures. The main IC on this board is the **BQ79616-Q1 battery monitor**, which is designed for multi-cell battery pack monitoring.

5.1 Cell Voltage Inputs

The CMU connects to the battery cells through the cell input connector. Each cell tap is routed through filtering and balancing circuitry before reaching the BQ79616-Q1 measurement pins.

The cell input filters use resistors and capacitors on the voltage sense lines. These filters help:

- Reduce measurement noise.
- Stabilize cell voltage readings.
- Protect the monitor inputs from small transients.

Accurate cell voltage measurement is one of the most important BMS functions. The cell input connector, sense-line order, and filtering network should be reviewed carefully before the CMU is connected to real cells.

5.2 Cell Balancing Support

The CMU includes hardware support for **cell balancing**. Balancing allows the BMS to reduce voltage differences between cells in a series battery pack.

This design provides the hardware foundation for balancing. The balancing resistor values, thermal behavior, BQ79616-Q1 balancing configuration, and firmware control logic still need to be verified before balancing is used with a real accumulator.

5.3 Temperature Sensing

The CMU includes **NTC thermistor inputs** for battery temperature monitoring. Thermistors can be placed near selected cells or key thermal locations in the accumulator.

The BMU can use these readings to detect overtemperature conditions and report thermal faults to the ECU.

5.4 CMU Communication Path

The CMU communicates with the BMU through the BQ79616-Q1 and BQ79600-Q1 communication path. The cell monitoring path uses a **daisy-chain structure**, which is more scalable than routing every cell directly to the STM32.

The design also includes **transformer-based isolated daisy-chain communication sections**. This allows the battery-side cell monitoring circuit to communicate with the BMU while keeping the battery-side circuitry separated from the low-voltage controller electronics.

6. Isolated Pack Current Sense Board Design

The **Isolated Pack Current Sense Board**, referred to as the **Shunt Board**, measures pack current through an external shunt resistor. Current flowing through the shunt resistor creates a small voltage drop. The board uses the **INA238-Q1 current, voltage, and power monitor** to measure this voltage drop and report current data to the BMU.

6.1 Current Measurement

The INA238-Q1 measures the voltage across the external shunt resistor and provides current, voltage, and power data through a digital I2C interface.

This approach allows the BMU to receive pack current data without directly measuring the shunt voltage with the STM32 ADC.

6.2 Isolated I2C Communication

The INA238-Q1 communicates over I2C. Since the Shunt Board is connected near the battery current path, the measurement side should not directly share the same ground reference as the BMU controller side.

The design uses the **ISO1540-Q1 isolated I2C interface** for SDA and SCL. This allows the STM32 on the BMU to read current data while maintaining electrical separation between the current sensing side and the BMU side.

6.3 Isolated ALERT Signal

The Shunt Board includes an **ISO7710-Q1 digital isolator** for the ALERT signal.

The ALERT signal can be used by the INA238-Q1 to notify the BMU of a current-related threshold event or

fault condition. By isolating this signal, the design preserves the safety boundary between the current sensing side and the BMU side.

6.4 Isolated Power Domain

The Shunt Board uses an isolated DC-DC converter, **NXF1S0303MC**, to generate an isolated 3.3 V rail for the current sensing side.

The BMU provides **3.3 V / GND** to the board on the controller side. The isolated DC-DC converter generates a separate **3.3VA / GNDA** domain for the INA238-Q1 measurement side.

This means:

- **3.3 V / GND** belongs to the BMU low-voltage controller side.
- **3.3VA / GNDA** belongs to the isolated current sensing side.
- These domains are **not the same electrical node**.
- GND and GNDA should not be shorted unless the isolation strategy is intentionally changed and revalidated.

6.5 Debugging Features

The Shunt Board includes:

- I2C pull-up resistors.
- ALERT pull-up circuitry.
- Decoupling capacitors near IC power pins.
- Power indication LEDs.
- Test points for power rails, I2C signals, ground references, and ALERT.

These features make the board easier to bring up and debug during subsystem testing.

7. Communication Design

The BMS uses different communication methods for different parts of the system.

Communication Path	Protocol / Signal	From	To	Isolation
CMU to BMU	Battery monitor daisy-chain communication	BQ79616-Q1	BQ79600-Q1	Transformer-based isolation
Shunt Board to BMU	I2C	INA238-Q1	STM32	ISO1540-Q1 isolated I2C

Shunt ALERT to BMU	ALERT digital signal	INA238-Q1	STM32	ISO7710-Q1 digital isolator
BMU to ECU	CAN / CAN-FD physical layer	STM32 / CAN transceiver	Vehicle ECU	Vehicle-side communication

The CMU communication path is used for cell voltage and temperature data. The BQ79616-Q1 collects battery-side measurements and sends them through the battery monitor communication path to the BQ79600-Q1 bridge on the BMU.

The Shunt Board communication path is used for current, voltage, and power data. The INA238-Q1 sends this data through isolated I2C. The ALERT signal is isolated separately because it may need to notify the BMU quickly when a current threshold or fault condition occurs.

The BMU communicates with the ECU over CAN. Once firmware and CAN message definitions are completed, the BMU can send cell voltages, pack current, battery temperature, fault status, and communication status to the vehicle ECU.

8. Power and Isolation Design

The power design is split into different voltage domains. This is one of the most important parts of the BMS design because the system includes both low-voltage controller electronics and battery-side sensing circuits.

Power Domain	Voltage	Ground Reference	Used By	Notes
BMU input	12 V	GND	BMU power input	Main low-voltage supply input
BMU logic rail	3.3 V	GND	STM32, logic circuits, communication circuits	Low-voltage controller side
Shunt isolated rail	3.3VA	GNDA	INA238-Q1 current sensing side	Generated by isolated DC-DC converter

Vehicle communication side	As designed for CAN transceiver	GND	CAN interface	Used for ECU communication

The BMU generates the regulated voltages needed for the STM32 and low-voltage communication circuits.

The Shunt Board uses isolated power because the current sensing side should not directly share the same ground as the BMU controller side. Although both sides use 3.3 V, they are separate electrical domains. The schematic separates these nets as **3.3 V / GND** on the BMU side and **3.3VA / GNDA** on the isolated measurement side.

Decoupling capacitors were placed near IC power pins to improve voltage stability and reduce noise. Power LEDs were included so the expected power rails can be checked quickly during bench testing. Test points were added to make it easier to measure power rails, communication signals, and ground references.

9. Monitoring and Fault Detection Logic

The BMS follows a layered monitoring and fault detection structure.

Step	Function	Main Board
1	Measure individual cell voltages and battery temperatures	CMU
2	Measure pack current through the external shunt resistor	Shunt Board
3	Collect voltage, temperature, and current data	BMU
4	Compare measurements against safety limits	STM32 on BMU
5	Set fault flags for unsafe conditions	STM32 on BMU
6	Send battery status and fault information to the ECU	BMU over CAN

This logic is important because the BMU does not directly measure every battery condition by itself.

Instead, the BMS uses dedicated sensing subsystems to collect battery data. The BMU then acts as the central decision-making and communication board.

The main fault categories are:

- **Overvoltage**
- **Undervoltage**
- **Overtemperature**
- **Overcurrent**
- **Communication errors**

The exact fault thresholds and response behavior should be defined during firmware integration and system validation.

10. Firmware Design

The firmware is designed around the STM32 microcontroller on the BMU. The STM32 initializes communication interfaces, collects battery data, checks measurements against safety limits, sets fault flags, and sends battery status to the ECU.

The main firmware tasks are:

1. Initialize STM32 peripherals, including I2C, CAN, GPIO, and the communication interface for the BQ79600-Q1.
2. Read individual cell voltages from the BQ79616-Q1 on the CMU.
3. Read temperature values from the NTC thermistor inputs on the CMU.
4. Read current, voltage, and power data from the INA238-Q1 on the Shunt Board.
5. Monitor the isolated ALERT signal from the current sensor.
6. Compare measured values against defined safety limits.
7. Set fault flags for overvoltage, undervoltage, overtemperature, overcurrent, or communication errors when detected.
8. Set the Shutdown and BMS LED signals when faults are detected.
9. Send battery data and fault information to the ECU over CAN.

Firmware should be tested in stages. The recommended bring-up order is:

1. Test STM32 programming through SWD.
2. Test basic CAN transmission.
3. Test I2C communication to the INA238-Q1.
4. Test isolated ALERT signal behavior.
5. Test BQ79600-Q1 to BQ79616-Q1 communication.
6. Test CMU voltage and temperature readings with safe simulated inputs.
7. Test full data collection and CAN reporting.

11. Design Rationale

The final design was chosen because it supports the main BMS functions while keeping the system **modular, isolated, and testable**.

The **BQ79616-Q1** was selected because it is designed for multi-cell battery monitoring. This makes it more appropriate than measuring every cell directly with the STM32.

The **BQ79600-Q1** was selected as the bridge between the STM32 and the BQ79616-Q1 battery monitor communication path.

The **INA238-Q1** was selected because it provides current, voltage, and power measurement through a digital I2C interface.

Isolation was a major design priority. The battery monitoring side and current sensing side can operate under different electrical conditions than the low-voltage BMU controller side. Isolated communication and isolated power help protect the STM32, CAN interface, and other vehicle-side electronics from wiring mistakes, ground reference differences, or unexpected voltage conditions during testing.

The design also includes practical bring-up features such as **test points, LEDs, connectors, and programming headers**. These features are important because each subsystem needs to be tested before the full BMS is connected together.

12. Design Limitations and Verification Needs

This design provides the core architecture for cell monitoring, temperature sensing, current sensing, CAN communication, and fault handling. However, it still requires hardware bring-up, firmware integration, and full validation before it can be used as a vehicle-ready BMS.

Before connecting the system to a real accumulator, the following items should be verified:

1. Final accumulator voltage and series cell count.
2. Required number of CMU boards.
3. Cell connector pinout and safe wiring order.
4. BMU, CMU, and Shunt Board power rails.
5. Isolation boundaries between **GND** and **GNDA**.
6. Isolated I2C communication to the INA238-Q1.
7. ALERT signal behavior through the ISO7710-Q1.
8. BQ79600-Q1 to BQ79616-Q1 communication.
9. CAN message format and ECU requirements.
10. Fault thresholds for overvoltage, undervoltage, overtemperature, and overcurrent.
11. Balancing circuit behavior and thermal performance.

Overall, the final design provides a modular foundation for the VMS BMS. The CMU measures cell voltage and temperature, the Shunt Board measures pack current, and the BMU collects the data, checks for faults, and prepares battery information for vehicle communication. This structure allows the system to be brought up one subsystem at a time and continued by future Viking Motorsports teams.

Test Plan Overview

Our test plan consists of a top-down test and the bottom-up subsystem tests. These tests will test Overvoltage, Undervoltage, and Overcurrent and the system will set the signal for the Shutdown Circuit and report each specific fault to the ECU with which specific cell(s) had the fault. The full Test plan is in the Appendix D.

Results

Project Result Summary

The project achieved its main objective at the prototype level. The team completed the BMS hardware design for the **Battery Management Unit (BMU)**, **Cell Monitoring Unit (CMU)**, and **Isolated Pack Current Sense Board**, referred to as the **Shunt Board** in this report.

The PCB designs were completed, reviewed, and tested on a small-scale prototype vehicle platform. Based on the testing completed so far, the overall hardware and software direction appears to work as intended, and no major system-level issues were found. The prototype successfully demonstrated the core BMS concept, but it has not yet been scaled to the full 600 V accumulator architecture.

The current system uses one CMU set. Future teams will need to expand the design to support the full battery stack, improve mechanical integration, and validate the system on the real vehicle.

Deliverables Achieved

Deliverable	Result
BMU design	Completed. The BMU provides main control, power regulation, CAN communication, BQ79600-Q1 bridge support, programming/debug access, and board-to-board connections.
CMU design	Completed. The CMU supports cell voltage monitoring, temperature sensing, balancing support, and BQ79616-Q1 battery monitor communication.
Shunt Board design	Completed at the prototype level. The board supports pack current sensing through an external shunt resistor, INA238-Q1 current monitor, isolated I2C, isolated ALERT, and isolated power.
PCB design and testing	Completed at the prototype level. The boards were reviewed and tested, and no major PCB issues were found during initial testing.
Small-scale prototype test	Completed. The system was tested on a small prototype vehicle platform to validate the basic BMS concept.
Documentation and handoff materials	Completed. The project provides design notes, schematic documentation, BOM information, and subsystem explanations for future teams.

Test Results and Design Notes

Prototype testing showed that the basic BMS architecture is functional at small scale. The BMU, CMU, and Shunt Board structure gives future teams a clear path to continue the project without restarting the architecture from scratch.

One layout issue was identified on the Shunt Board. The isolated DC-DC converter is physically large and blocks access to **C6**, so this area should be rearranged in the next PCB revision. This does not invalidate the current design, but it should be corrected before the next board version.

The board-to-board interfaces should also be improved. The BMU, CMU, and Shunt Board connectors work for prototype testing, but future revisions should improve connector placement, plug-in direction, cable access, and ease of assembly. This will make the system easier to test, maintain, and install in the real vehicle.

Requirement Satisfaction

Requirement	Result
Cell voltage monitoring	Met at prototype scale. The CMU supports cell voltage monitoring, but full accumulator validation requires multiple CMUs.
Temperature monitoring	Met at prototype scale. The CMU includes NTC thermistor inputs; vehicle-level thermal validation remains future work.
Pack current measurement	Met at prototype scale. The Shunt Board supports isolated pack current sensing; the layout should be revised before the next version.
Cell balancing support	Hardware support included. Balancing behavior and thermal performance still need full validation before use with real cells.
CAN / ECU communication	Supported by the BMU design. Final CAN message definitions and full ECU integration remain future work.
Isolation strategy	Supported by the design through isolated current sensing communication and isolated power. Full vehicle-level isolation validation remains future work.
Vehicle-ready 600 V BMS	Not fully met. The project produced a working prototype-level BMS design, not a fully validated 600 V vehicle-ready system.

Overall Result

Overall, the project met its main purpose: it turned the VMS BMS from a broad concept into a completed prototype-level design that future teams can understand, reproduce, and continue.

The main remaining work is not basic circuit design, but **scale-up and vehicle integration**. The next

team should expand the system from one CMU to the full 600 V accumulator architecture, revise the Shunt Board layout around the DC-DC converter and C6, improve board-to-board connector usability, complete final firmware and CAN-FD integration, and work with the mechanical team to package the BMS safely inside the actual race car.

Post Mortem

What Worked

The overall BMS design work was completed successfully. The team finished the main hardware architecture for the BMU, CMU, and Shunt Board. The PCB designs were reviewed and tested at the prototype level, and no major PCB issues were found during initial testing.

The modular architecture worked well. Separating the system into the BMU, CMU, and Shunt Board made the design easier to understand, debug, and improve. This structure also gives future teams a clear path to scale the system from a small prototype to a full accumulator-level BMS.

The prototype test platform was also useful. Testing with a toy car / small prototype setup allowed the team to check the basic BMS concept before moving toward a real high-voltage vehicle system.

What Did Not Work

The project did not reach full vehicle-level validation. The current prototype only uses one CMU set, so it does not yet represent the full 600 V accumulator architecture. A real vehicle-ready BMS will require multiple CMUs, full battery-stack communication, complete firmware integration, and more safety validation.

The project also did not complete mechanical and vehicle integration. The BMS still needs to be placed inside the real vehicle, which requires coordination with the mechanical team. Board mounting, available space, wiring paths, connector access, vibration, cooling, electrical noise, and high-voltage separation still need to be considered.

What We Would Do Differently

We would plan the scale-up path earlier. The prototype proved the basic design direction, but the next major challenge is expanding from one CMU to a full 600 V accumulator system.

We would also involve the mechanical and vehicle integration teams earlier. A working PCB is not enough for a race car. The system also needs a safe physical location, stable mounting, clean wiring, connector access, and protection from vibration, heat, and electrical interference.

What We Wish We Knew Earlier

We wish we had known earlier how much work comes after the circuit design is complete. Finishing the schematic, PCB, and prototype testing is only one stage of the BMS project. Turning the prototype into a vehicle-ready system requires scaling, firmware completion, mechanical packaging, safety validation, and full vehicle testing.

Future Work

The next team should treat this project as a completed prototype-level BMS design, not a finished vehicle-ready system. The main technical task is to scale the architecture from one CMU to the full number of CMUs required for the real 600 V accumulator. This includes checking the final series cell count, verifying the daisy-chain communication path, and confirming that the BMU can collect data from the full battery stack.

The firmware also needs to move from prototype support to full system operation. The next team should complete voltage, temperature, current, fault detection, and CAN reporting functions. They should also define final CAN message IDs, data format, update rate, and fault thresholds with the ECU team. Both the BMS and ECU teams should find a USB-to-CAN converter that can read CAN-FD and use the original BMS CAN-FD Database after the switch.

Mechanical integration is the other major next step. The BMS boards need to be placed inside the actual vehicle architecture with proper mounting, wiring, connector access, cooling, and separation from high-voltage components. The next team should work closely with the mechanical team to avoid space, vibration, cable routing, and interference problems.

Before vehicle testing, the team should validate the full system under realistic accumulator-level conditions. The goal should be to prove that the BMU, multiple CMUs, Shunt Board, firmware, CAN communication, and safety logic all work together before the BMS is installed on the real car.

Project Resources

The BMS firmware, schematics, and PCB layout are on a GitHub repository called **Battery-Management-System-Capstone-2026** under the Viking Motorsports Organization. The BOMs are included on the KiCAD projects for each module.

The Documentation for the project including the final report will be the VMS Google Drive under the ECE folder called VMS BMS capstone drive.

Conclusion

Our capstone project was on developing a brand new Battery Management System from scratch for Viking Motorsports' EV in the EV FSAE competition. After some background research, we determined to use similar modular system architecture to Libre Solar BMS as well as the components found on their CMU to start off our design. We managed to create the system architect design and have a successful prototype made for our system using evaluation boards. The only snag for us was ordering and receiving boards from OSH Park took longer than expected leaving us no time to fully test if the boards worked properly. Regardless, we are now ready to hand off this project to future capstone teams and we hope that this report, its appendices and the other documentation were able to explain to you why we design the BMS this way.

Appendix A: Tooling

This appendix describes the tools required to build, configure, program, and operate the final Battery Management System prototype. This section focuses on the final prototype tooling only.

A.1 Final Prototype Hardware Tools

Table A.1 lists the major hardware tools and platforms required to operate and reproduce the final prototype.

Tool / Platform	Version / Model	Purpose
STM32 Nucleo-C092RC	STM32 Nucleo-C092RC	Main embedded controller used to run the BMS firmware.
TI BQ79600EVM	TI BQ79600EVM	Bridge device used to communicate between the STM32 controller and the BQ79616 battery monitor.
TI BQ79616EVM	TI BQ79616EVM	Battery monitor evaluation module used to monitor the 6S $LiFePO_4$ pack.
TI INA238EVM	TI INA238EVM	Current and voltage measurement evaluation module used with the system shunt.
CAN to USB converter	Copperforge vulCAN	Converts STM32 CAN signals to USB.
Computer	Windows, macOS, or Linux	Used to install the firmware toolchain, build the firmware, flash the STM32 Nucleo, and view serial logs.

A.2 Final Prototype Software Tools

Table A.2 lists the software tools required to build and flash the final firmware.

Tool / Platform	Version / Model	Purpose
Git	Latest	Used to clone and manage the firmware repository.
Python	Latest	Required for the Zephyr development environment and west.
West	Latest	Zephyr command-line tool used to initialize the workspace, build the firmware, and flash the board.
Zephyr RTOS	Latest	Embedded real-time operating

		system used by the BMS firmware.
Zephyr SDK	Latest	Cross-compilation toolchain used to build the STM32 firmware.
CMake	Latest	Build configuration tool used by Zephyr.
Ninja	Latest	Build backend used by Zephyr.
STM32CubeProgrammer	Latest	Flashing utility used by west to program the STM32 Nucleo.

A.3 Zephyr Toolchain Installation

The BMS firmware is built using Zephyr RTOS. To install the Zephyr development environment, follow the official Zephyr Getting Started Guide for the operating system being used. The guide provides step-by-step instructions for installing system dependencies, creating a Python virtual environment, installing west, initializing a Zephyr workspace, installing the Zephyr SDK, and installing Python dependencies.

Official setup reference: Zephyr Project Getting Started Guide.

After completing the Zephyr installation, make sure to flash the blinky program on the STM32 Nucleo-C092RC as a final verification that everything is set up to build and flash to the board correctly.

Appendix B: Developer's Manual

This appendix describes the Battery Management System prototype from the developer's point of view. It is intended for future developers who need to understand, modify, program, test, or troubleshoot the system.

B.1 Developer Overview

The Battery Management System prototype monitors a 6S $LiFePO_4$ battery pack and transmits battery data to an external ECU over CAN. The system uses an STM32 Nucleo-C092RC as the main controller, a TI BQ79600EVM as the bridge device, a TI BQ79616EVM as the cell monitor, and a TI INA238EVM for current measurement through an external low-side shunt.

The STM32 firmware performs the following main functions:

- Initializes the CAN interface.
- Initializes the I2C interface for the INA238EVM.
- Wakes the BQ79600EVM bridge device.
- Initializes and auto-addresses the BQ79616EVM stack monitor.
- Reads individual cell voltages.
- Reads temperature values.
- Reads current and pack electrical data.

- Reads and clears battery monitor faults.
- Runs balancing logic, if enabled.
- Transmits BMS telemetry over Classic CAN.

B.2 System Architecture

The prototype is organized into the following major subsystems:

Subsystem	Main Hardware	Purpose
Main Controller	STM32 Nucleo-C092RC	Runs BMS firmware and coordinates all measurements and CAN telemetry.
Bridge Interface	TI BQ79600EVM	Provides communication bridge between STM32 UART and BQ79616EVM stack device.
Cell Monitoring	TI BQ79616EVM	Measures cell voltages, temperatures, faults, and balancing status.
Current Sensing	TI INA238EVM and Shunt Resistor	Measures pack/load current through a low-side shunt.
CAN Communication	CAN Transceiver Module	Sends BMS data to the external ECU using CAN_H and CAN_L.
Battery Pack	6S $LiFePO_4$ pack	Energy source monitored by the BMS.

B.3 Hardware Configuration

B.3.1 Battery Pack Configuration

The prototype uses a 6S $LiFePO_4$ battery pack. The battery nodes are labeled B0 through B6.

Node	Description
B0	Cell 1 Negative / Pack Negative
B1	Cell 1 Positive
B2	Cell 2 Positive
B3	Cell 3 Positive
B4	Cell 4 Positive
B5	Cell 5 Positive

B6	Cell 6 Positive / Pack Positive

Each $LiFePO_4$ cell has a nominal voltage of approximately 3.20 V and a maximum charge voltage of 3.65 V. Therefore, the maximum 6S pack voltage is 21.90 V.

B.3.2 BQ79616EVM 6S Configuration

The BQ79616EVM supports up to 16 cell inputs, but this prototype uses only six cells. The actual battery cell nodes B0 through B6 are connected to the corresponding lower cell inputs. Unused upper cell inputs are tied to the top-of-stack node so that they are not left floating.

For the final 6S configuration:

- B0 connects to the BQ79616EVM bottom cell reference.
- B1 through B6 connect to the first six cell monitor inputs.
- Unused upper cell channels are shorted to B6/top-of-stack.
- The shorting configuration must be verified before applying battery power.

Before operating the system, verify adjacent cell voltages with a multimeter. Each adjacent tap should measure one cell voltage, and the total B0-to-B6 voltage should equal the sum of the six cells.

B.3.3 BQ79600EVM to STM32 Nucleo UART Connection

The STM32 communicates with the BQ79600EVM using UART.

STM32 Signal	STM32 Pin	BQ79600EVM Signal	Purpose
USART1_TX / Wake	PB6	MOSI_RX	Sends wake pulse and UART commands to BQ79600EVM.
USART1_RX	PB7	MISO_TX	Receives UART responses from BQ79600EVM.
GND	GND	GND	Shared electronics reference.
VIO / logic supply	3.3V	VIO	Logic reference for communication.

The firmware generates the BQ79600 wake pulse before initializing UART communication. If BQ communication fails, verify the wake signal, UART TX line, UART RX line, and board power.

B.3.4 INA238EVM to STM32 Nucleo I2C Connection

The INA238EVM communicates with the STM32 over I2C.

STM32 Signal	STM32 Pin	INA238EVM Signal	Purpose
I2C1_SCL	PB8	SCL	I2C clock.
I2C1_SDA	PB9	SDA	I2C data.
3.3V	3V3	VS / supply	INA238EVM logic power.
GND	GND	GND	Shared electronics reference.

The INA238 measures current by reading the voltage drop across the external shunt resistor. The shunt polarity must match the firmware current sign convention.

B.3.5 Shunt Configuration

The prototype uses a low-side shunt resistor for current measurement. The shunt is installed in the selected load return path.

General low-side configuration:

Node	Description
Battery Side of Shunt	Connected to pack negative side.
Load Side of Shunt	Connected to load/system return side.
INA238EVM Shunt Sense Negative	Connected to battery side of shunt.
INA238EVM Shunt Sense Positive	Connected to load side of shunt.

The final system configuration determines which current is measured. If only the XT60 return path passes through the shunt, then the INA238 measures the current flowing through that XT60 load path. Any electronics current that bypasses the shunt is not included in the measured current.

B.3.6 CAN Transceiver Connection

The STM32 communicates with the external ECU using a CAN transceiver.

STM32 Signal	CAN Transceiver Signal	Purpose
CAN_H	CAN_H	CAN bus high line.
CAN_L	CAN_H	CAN bus low line.
GND	GND	CAN transceiver reference.

The final CAN bus uses Classic CAN with standard 11-bit identifiers and payloads of 8 bytes or fewer.

CAN_H and CAN_L should be routed as a twisted pair. Termination should be 120 Ω at the two physical ends of the CAN bus.

B.3.7 Power Distribution

The battery pack powers the electronics through a buck converter. The buck converter steps pack voltage down to 5 V.

Power Rail	Source	Loads
Pack Voltage	6S $LiFePO_4$ battery pack	XT60 output, buck converter input, BQ79616EVM cell monitor connection.
5V	Buck converter output	STM32 Nucleo, BQ79600EVM.
3.3V	STM32 Nucleo regulator	INA238EVM and logic-level peripherals.

B.4 Firmware Architecture

The firmware is organized into the following subsystem modules:

Firmware Area	Responsibility
Main application	Calls initialization functions and runs the main monitoring loop.
Monitor initialization	Main loop that initializes CAN, I2C, wake GPIO, UART, BQ wake, and BQ auto-addressing.
Voltage module	Reads individual cell voltages from the BQ79616EVM.
Temperature module	Reads temperature sensors from the BQ79616EVM.
Current module	Configures and reads the INA238EVM over I2C.
Fault module	Reads, reports, and clears BQ79600/BQ79616 fault registers.
Balancing module	Determines cell balancing behavior based on cell voltage differences.
Telemetry module	Packs and transmits BMS data over CAN.
Debug module	Prints all values to the serial monitor for easier debugging.
CAN module	Initializes CAN bus.

I2C module	Initializes I2C bus.
UART module	Initializes UART module.
Metrics module	Calculates battery metrics to send over CAN bus.
Protocols module	Define communication protocols between hardware devices in the system architecture.

B.5 Firmware Initialization Flow

The expected firmware startup sequence is:

1. Initialize CAN.
2. Initialize I2C.
3. Configure the BQ79600 wake GPIO.
4. Generate the BQ79600 wake pulse.
5. Switch the wake/UART pin to UART alternate function.
6. Initialize UART communication.
7. Send BQ wake command through the BQ79600EVM.
8. Auto-address the BQ79616EVM stack device.
9. Initialize voltage monitoring.
10. Initialize temperature monitoring.
11. Initialize current monitoring.
12. Initialize fault monitoring.
13. Initialize balancing logic.
14. Begin the main monitoring and CAN telemetry loop.

If initialization fails, check the serial log to determine which subsystem failed first.

B.6 Building and Flashing Firmware

Tool installation and Zephyr environment setup are documented in Appendix A. After the toolchain is installed, use the following project-specific build and flash commands.

Clone the BMS firmware repository and change to the appropriate directory:

```
git clone
git@github.com:VikingMotorsports/Battery-Management-System-Capstone.git
cd Battery-Management-System-Capstone/Software/bms_monitor
```

Source the corresponding Python virtual environment:

```
source [insert venv activate script]
```

Build the firmware:

```
west build -p always -b nucleo_c092rc
```

Connect the STM32 Nucleo-C092RC to the computer using USB. Then run:

```
west flash
```

Open the serial monitor after flashing to verify firmware startup logs.

B.7 CAN Message Summary

The final firmware transmits BMS data using Classic CAN with standard 11-bit identifiers. Each message uses 8 bytes or fewer.

Message	CAN ID	Payload
BMS_Protection	0x010	Protection and fault flags.
BMS_CellVoltageMin	0x250	Minimum cell ID and minimum cell voltage.
BMS_CellVoltageMax	0x251	Maximum cell ID and maximum cell voltage.
BMS_CellTemperatureMin	0x252	Minimum temperature sensor ID and minimum temperature.
BMS_CellTemperatureMax	0x253	Maximum temperature sensor ID and maximum temperature.
BMS_PackElectrical	0x254	Pack current and pack voltage.
BMS_PackPower	0x255	Pack power.

Appendix C: User's Manual

This appendix explains how to operate the Battery Management System prototype from the system user's point of view. The prototype monitors a 6S $LiFePO_4$ battery pack, reads cell voltage, temperature, current, and fault information, and transmits battery data to an external ECU over CAN.

C.1 System Overview

The Battery Management System prototype consists of the following major hardware boards:

- STM32 Nucleo-C092RC main controller
- TI BQ79600EVM bridge board
- TI BQ79616EVM battery monitor board
- TI INA238EVM current measurement board
- CAN transceiver module
- 6S $LiFePO_4$ battery pack
- Low-side shunt resistor

The STM32 Nucleo runs the BMS firmware. The firmware wakes and configures the BQ79600/BQ79616 battery monitor system, reads battery measurements, reads current through the INA238, checks for faults, and sends telemetry over CAN.

C.2 Prototype Connections

Table C.1 describes the external connections used by the prototype.

Connector / Interface	Purpose
XT60 Female Connector	Main battery output connection for the external load system.
CAN_H and CAN_L	CAN communication lines between the BMS and the external ECU.
STM32 Nucleo USB port	Used to flash firmware and view serial logs.
Battery Cell Harness	Connects the 6S battery cell nodes to the BQ79616EVM. This harness should already be installed in the prototype.
JST-XH Female Connector	Provides regulated power to the STM32 Nucleo, BQ79600EVM, and INA238EVM.
Molex Connector	Connects the STM32 Nucleo to the BQ79600EVM for communication.
Molex Connector	Connects the STM32 Nucleo to the INA238EVM for communication and power.
COMH and COML	Connects the BQ79600EVM to the BQ79616EVM for isolated daisy-chain communication.

The final prototype is intended to be operated using the completed harnesses and connectors. Users should not need to rebuild the harnesses during normal operation.

C.3 Hardware Connection Procedure

Before powering the system, confirm that all boards and connectors are connected correctly.

C.3.1 Battery Pack and BQ79616EVM

1. Confirm that the 6S $LiFePO_4$ battery pack is installed in the battery box.
2. Confirm that the 7-wire cell sense harness is connected to the BQ79616EVM.
3. Confirm that the harness is connected in cell order from B0 through B6:
    - B0: pack negative
    - B1: cell 1 positive
    - B2: cell 2 positive
    - B3: cell 3 positive

- B4: cell 4 positive
- B5: cell 5 positive
- B6: pack positive

C.3.2 BQ79600EVM and BQ79616EVM

1. Confirm that the BQ79600EVM is connected to the BQ79616EVM using the required COMH and COML isolated daisy-chain connector.
2. Confirm that the BQ79600EVM is powered from the regulated 5 V electronics rail.
3. Confirm that the BQ79616EVM is connected to the battery pack through the cell sense harness.

C.3.3 STM32 Nucleo Connections

1. Confirm that the STM32 Nucleo-C092RC is connected to the BQ79600EVM through the corresponding molex connector.
2. Confirm that the STM32 Nucleo is connected to the INA238EVM over through the corresponding molex connector.
3. Confirm that the STM32 Nucleo is connected to the CAN transceiver module.
4. Confirm that the STM32 Nucleo receives regulated electronics power from the battery using the corresponding JST-XH connector.

C.3.4 INA238EVM and Shunt

1. Confirm that the INA238EVM is connected across the low-side shunt resistor.
2. Confirm that the shunt is installed in the intended load current path.

C.3.5 CAN Connection to External ECU

1. Connect BMS CAN_H to external ECU CAN_H.
2. Connect BMS CAN_L to external ECU CAN_L.

C.4 Firmware Build and Flash Procedure

The firmware must be built and flashed to the STM32 Nucleo-C092RC before operating the prototype. The Zephyr toolchain setup is described in Appendix A.

C.4.1 Build the Firmware

The firmware must be built and flashed to the STM32 Nucleo-C092RC before operating the prototype. The Zephyr toolchain setup is described in Appendix A.

Clone the BMS firmware repository and change to the appropriate directory:

```
git clone
git@github.com:VikingMotorsports/Battery-Management-System-Capstone.git
cd Battery-Management-System-Capstone/Software/bms_monitor
```

Source the corresponding Python virtual environment:

```
source [insert venv activate script]
```

Build the firmware:

```
west build -p always -b nucleo_c092rc
```

A successful build creates the firmware output in the build/ directory.

C.4.2 Flash the Firmware

Connect the STM32 Nucleo-C092RC to the computer using USB. Then run:

```
west flash
```

C.4.3 Serial Monitor Configuration

After flashing, open a serial terminal to view firmware logs. If using VSCode, click on the ‘serial monitor’ tab after opening the integrated terminal window. Select the correct serial port in the corresponding drop down menu and then click ‘start monitoring’. If using another serial monitor viewer tool, follow the corresponding steps to configure that tool.

Appendix D: Test Plan

This document lays out how someone is able to ensure the BMS is fully operational. I recommend that you do Bottom up testing before Top Down.

D.1 Top Down Testing

D.1.1 Purpose

To test the entire system against its requirements.

D.1.2 Equipment Needed

- System Equipment
    - Battery Monitoring Module
    - BMU (Battery Management Unit)
    - Current SHUNT Module
    - 1.5 milliOhm SHUNT Resistor
    - 12V battery with LV system power bus
    - Jumper wires for Daisy chain and I2C interfaces
- Test Equipment
    - CAN-to-USB connector
    - Hair Dryer
    - Multimeter
    - Power Supply
- Other Equipment
    - HV Load

D.1.3 Top Down Test Steps

1. Confirm power rails are correct.
2. Confirm the cell sense harness is connected.
3. Confirm the shunt and INA238 are connected.

4. Connect CAN_H and CAN_L to the external ECU.
5. Flash and run the firmware.
6. Confirm normal initialization in the serial log.
7. Confirm cell voltage, temperature, current, and fault readings.
8. Apply a controlled external load.
9. Confirm current measurement responds to load.
10. Confirm CAN telemetry updates during load.
11. Remove the load.
12. Shut the system down safely.

Pass condition: The system initializes, reports battery data, sends CAN telemetry, and responds correctly to load changes.

D.1.4 Post-Test Teardown

1. Remove LV power bus from BMU.
2. Remove all the connections between the modules.
3. Disassemble the HV system.
4. Remove the Current SHUNT module from SHUNT resistor and the Monitor from each battery module.

D.2 Bottom-Up Testing

Each section below shows the test for each subsystem.

D.2.1 Power Rail Test

Purpose: Verify that all required power rails are present before running firmware.

Procedure:

1. Disconnect the external load.
2. Power the battery pack and STM32 through the buck converter.
3. Measure the buck converter output.
4. Confirm the 5 V rail is approximately 5.0 V.
5. Power the BQ79600EVM.
6. Measure the STM32 3.3 V rail.
7. Confirm the 3.3 V rail is approximately 3.3 V.
8. Check that no board, wire, or connector becomes hot.

Pass condition: 5 V and 3.3 V rails are present and stable.

D.2.2 Cell Sense Harness Test

Purpose: Verify that the battery harness is connected in the correct order.

Procedure:

1. Disconnect the harness from the BQ79616EVM if needed.
2. Measure voltage between adjacent cell sense wires.
3. Confirm each adjacent measurement is one cell voltage.

4. Measure from B0 to B6 and confirm total pack voltage.
5. Confirm no adjacent sense wires are shorted together.
6. Reconnect the harness only after the pinout is verified.

Pass condition: Each adjacent tap reads one cell voltage, and B0-to-B6 equals total pack voltage.

D.2.3 Cell Sense Harness Test

Purpose: Verify that the STM32 can wake and communicate with the BQ monitor system.

Procedure:

1. Power the BQ79600EVM and BQ79616EVM.
2. Flash the STM32 firmware.
3. Open the serial monitor.
4. Confirm that the bridge wake sequence completes.
5. Confirm that auto-addressing completes.
6. Confirm that six cell voltages are reported.
7. Compare reported cell voltages to multimeter measurements.

Pass condition: Firmware initializes the BQ stack and reports reasonable cell voltages for all six cells.

D.2.4 INA238 Current Measurement Test

Purpose: Verify current sensor operation and shunt polarity.

Procedure:

1. Confirm the INA238EVM is powered from 3.3 V.
2. Confirm I2C wiring is connected.
3. Run the firmware with no external load.
4. Confirm current reading is near zero.
5. Apply a known load.
6. Confirm current reading increases in the expected direction.
7. Compare firmware current reading to an external measurement or expected load current.

Pass condition: Current measurement changes correctly with load and has the expected polarity.

D.2.5 CAN Communication Test

Purpose: Verify that BMS telemetry is transmitted to the external ECU or CAN monitoring tool.

Procedure:

1. Connect CAN_H to CAN_H.
2. Connect CAN_L to CAN_L.
3. Confirm CAN termination is installed only at the two physical ends of the bus.
4. Power the BMS and external ECU.
5. Run the firmware.
6. Observe received CAN messages.

7. Verify message IDs, payload lengths, and signal values.

Pass condition: External ECU or CAN monitor receives valid BMS CAN messages.